

COMPENDIO TÁCTICO: RAMIFICACIÓN Y PODA

Branch & Bound: El huevo de Cristal y Bellido lo mama

1. El Eje Teórico: Backtracking vs Ramificación y Poda (RyP)

Mientras que el Backtracking busca *cualquier* solución válida mediante recursividad (DFS - Profundidad), RyP está diseñado exclusivamente para problemas de **Optimización** (mínimos o máximos). Utiliza una estructura de datos iterativa para explorar el árbol inteligentemente.

El Concepto Clave: Las Cotas (La Bola de Cristal)

Para no explorar todo el árbol, RyP predice el futuro de un nodo incompleto usando dos valores:

- **Cota Pesimista (El Récord Actual):** El valor de la mejor solución completa que hemos encontrado hasta ahora. Si minimizamos, empezamos en ∞ .
- **Cota Optimista (La Esperanza):** Una función matemática rápida que estima "lo mejor que podría llegar a ser" este nodo si lo completamos. **Si la Esperanza es PEOR que el Récord, podemos el nodo de raíz.**

2. Metodología de Resolución: Los 4 Pilares Innegociables

En todo examen de diseño de RyP, tienes que definir estos 4 elementos antes de programar:

- **1. Ramificación (Cómo nacen los hijos):** ¿Qué decisión tomo en cada nivel? (Ej: Meter o no meter el objeto i en la mochila).
- **2. Acotación (La Bola de Cristal):** ¿Cómo calculo la Cota Optimista del nodo actual? (Ej: Usar el algoritmo Voraz fraccionario para ver el beneficio máximo teórico).
- **3. Estrategia de Exploración (El GPS):** ¿En qué orden proceso los nodos vivos?
 - *FIFO / BFS (Cola normal):* A lo ancho. Malo para optimizar.
 - *LIFO / DFS (Pila):* En profundidad (Como Backtracking). Encuentra récords rápido.
 - *LC (Least Cost / Cola de Prioridad):* El Rey de RyP. Siempre saco el nodo con la mejor cota optimista.
- **4. Criterios de Poda (La Guillotina):** Condición exacta para descartar un nodo (Ej: si minimizo, corto si $CotaOptimista(Nodo) \geq MejorSolucion$).

3. El Motor Único: Plantillas "Llegar y Sustituir"

En Ramificación y Poda no hay recursividad, hay un bucle `while` y una Cola de Prioridad (Montículo). Aquí tienes las 3 plantillas exactas para los problemas de examen y sus herramientas de análisis.

A. Patrón "El Contrabandista" (Mochila 0/1 - MAXIMIZAR)

- **Disparador en Examen:** "Maximizar beneficio/valor", "Capacidad máxima", "Objetos indivisibles".
- **El Truco del 10:** Ordena siempre los objetos por la ratio b_i/p_i (beneficio/peso) de forma descendente ANTES de empezar.
- **Cola de Prioridad:** Max-Heap (El nodo con MAYOR cota sale primero).

Cartucho de Reconstrucción: ¿Qué objetos he metido?

Para devolver no solo el beneficio máximo, sino la lista de objetos elegidos:

- **En la estructura del Nodo:** Añade un `vector<int> solucion` (o array de booleanos).
- **Al crear el Hijo 1 (SÍ lo meto):** Hereda el vector del padre y haz `h1.solucion.push_back(1)`.
- **Al crear el Hijo 2 (NO lo meto):** Hereda el vector del padre y haz `h2.solucion.push_back(0)`.
- **Meta:** Cuando actualices `mejor_ben`, guarda también `mejor_solucion = h.solucion`.

Plantilla A: Mochila 0/1

```
int ryp_mochila() {
    MaxHeap vivos;
    int mejor_ben = 0; // COTA PESIMISTA: Récord a batir (Maximizar)

    // CARTUCHO 1: Raíz
    Nodo raiz = {0, 0, 0, 0, {}};
    raiz.cota = cota_voraz_fraccional(raiz); // Capacidad restante con fracciones
    vivos.insertar(raiz);

    while (!vivos.vacio()) {
        Nodo actual = vivos.extraer_max();

        // CARTUCHO 2: La Guillotina (Si mi esperanza es peor que la realidad, muero)
        if (actual.cota <= mejor_ben) continue;

        int i = actual.nivel + 1; // Siguiente objeto

        // HIJO 1: SÍ METO EL OBJETO
        if (actual.peso_actual + p[i] <= CAPACIDAD) { // Poda de viabilidad
            Nodo h1 = {i, actual.ben_actual + b[i], actual.peso_actual + p[i], 0, actual.solucion};
            h1.solucion.push_back(1); // RASTRO
            h1.cota = cota_voraz_fraccional(h1);

            if (h1.nivel == N && h1.ben_actual > mejor_ben) {
                mejor_ben = h1.ben_actual;
                mejor_solucion = h1.solucion;
            } else if (h1.cota > mejor_ben) vivos.insertar(h1);
        }

        // HIJO 2: NO METO EL OBJETO
        Nodo h2 = {i, actual.ben_actual, actual.peso_actual, 0, actual.solucion};
        h2.solucion.push_back(0); // RASTRO
        h2.cota = cota_voraz_fraccional(h2);

        if (h2.nivel == N && h2.ben_actual > mejor_ben) {
            mejor_ben = h2.ben_actual;
            mejor_solucion = h2.solucion;
        } else if (h2.cota > mejor_ben) vivos.insertar(h2);
    }
    return mejor_ben;
}
```

B. Patrón .^{EI} Rutero”(Viajante de Comercio TSP - MINIMIZAR)

- **Disparador en Examen:** Ruta más corta”, ”Visitar todas las ciudades sin repetir y volver al origen”, Ciclo de coste mínimo”.
- **Cola de Prioridad:** Min-Heap (El nodo con MENOR cota sale primero).

El ”Sabotaje”de la Matriz (La Bola de Cristal)

La cota es la suma de los mínimos que se restan a filas y columnas. Para predecir el coste de viajar de $i \rightarrow j$:

1. **Hucha Base:** Partes de la cota del padre + `MatrizPadre[i][j]` (el coste del billete).
2. **Quemar Naves:** En la nueva matriz pones Fila $i = \infty$ (ya saliste) y Columna $j = \infty$ (ya llegaste).
3. **Evitar Bucles:** Pones `MatrizHijo[j][1] =  $\infty$` (prohibido volver a casa pronto).
4. **Nueva Reducción:** Buscas los nuevos mínimos en las filas/columnas restantes, los restas y los sumas a la cota.

Plantilla B: Viajante de Comercio (Rutas/Grafos)

```
int ryp_viajante() {
    MinHeap vivos;
    int mejor_coste = INF; // COTA PESIMISTA: Récord a batir (Minimizar)

    Nodo raiz = iniciar_nodo_raiz();
    raiz.cota = reducir_matriz(raiz.matriz); // Suma de mínimos restados de filas y cols
    vivos.insertar(raiz);

    while (!vivos.vacio()) {
        Nodo actual = vivos.extraer_min();
        if (actual.cota >= mejor_coste) continue; // Guillotina

        if (actual.nivel == N) { // Llegada a Meta
            int ciudad_final = actual.ruta[N];
            if (matriz_original[ciudad_final][1] != INF) { // Coste de volver al inicio
                int coste_total = actual.cota + matriz_original[ciudad_final][1];
                if (coste_total < mejor_coste) mejor_coste = coste_total;
            }
            continue;
        }

        int origen = actual.ruta[actual.nivel];
        for (int destino = 2; destino <= N; destino++) { // Probar todas las ciudades
            if (!visitado(actual.ruta, destino) && actual.matriz[origen][destino] != INF) {
                Nodo hijo = crear_hijo(actual, destino);

                // SABOTAJE DE LA MATRIZ DEL HIJO
                anular_fila(hijo.matriz, origen);
                anular_columna(hijo.matriz, destino);
                hijo.matriz[destino][1] = INF;

                int coste_reduccion = reducir_matriz(hijo.matriz);

                // COTA = Lo inevitable + El Viaje + El impuesto del futuro
                hijo.cota = actual.cota + actual.matriz[origen][destino] + coste_reduccion;

                if (hijo.cota < mejor_coste) vivos.insertar(hijo);
            }
        }
    }
    return mejor_coste;
}
```

C. Patrón Recursos Humanos”(Asignación de Tareas - MINIMIZAR)

- **Disparador en Examen:** "Emparejar 1 a 1", " N trabajadores y N tareas", "Mínimo coste total".
- **La Cota (Matemática Rápida):** A diferencia del TSP, aquí no "saboteas" la matriz entera copiándola en cada nodo. Simplemente sumas lo que ya llevas gastado + el mínimo valor posible de cada trabajador que aún no tiene tarea asignada.

Plantilla C: Asignación de Tareas

```
int ryp_asignacion() {
    MinHeap vivos;
    int mejor_coste = INF;

    Nodo raiz = {0, 0, array_vacio, 0};
    raiz.cota = sumar_minimos_de_todas_las_filas(); // Esperanza hiper-optimista
    vivos.insertar(raiz);

    while (!vivos.vacio()) {
        Nodo actual = vivos.extraer_min();
        if (actual.cota >= mejor_coste) continue;

        int tarea_actual = actual.nivel + 1; // Siguiendo tarea a asignar

        for (int trab = 1; trab <= N; trab++) {
            if (!actual.trabajador_asignado[trab]) { // Poda de viabilidad
                Nodo hijo = clonar(actual);
                hijo.trabajador_asignado[trab] = true;
                hijo.nivel = tarea_actual;
                hijo.coste_acumulado += matriz_costes[tarea_actual][trab];

                // COTA: Lo que llevo pagado + los mínimos de las tareas restantes
                hijo.cota = hijo.coste_acumulado + sumar_minimos_filas(tarea_actual + 1, N);

                if (hijo.nivel == N) { // Meta
                    if (hijo.coste_acumulado < mejor_coste) mejor_coste = hijo.coste_acumulado;
                } else if (hijo.cota < mejor_coste) {
                    vivos.insertar(hijo);
                }
            }
        }
    }

    return mejor_coste;
}
```

4. Despliegue de los Cartuchos: Los Clásicos de RyP

A. El Problema de la Mochila 0/1 (Maximización)

- **Nodo:** Representa los objetos analizados hasta el momento (i), el beneficio actual y el peso actual.
- **Ramificación:** 2 hijos por nodo. El hijo izquierdo **METE** el objeto $i + 1$, el hijo derecho **NO LO METE**.
- **Estrategia:** *Best-First* (Cola de prioridad max-heap). Sacamos el nodo con la mayor cota optimista.
- **Poda Práctica:** Si el peso supera la capacidad, podemos por viabilidad. Si $Cota \leq Mejor_Solucion_Encontrada$, podemos por optimización.

B. Viajante de Comercio (TSP) (Minimización)

- **Nodo:** Tupla con el camino actual, coste acumulado y la *Matriz Reducida* del estado actual.
- **Ramificación:** Desde la ciudad A , generamos un hijo por cada ciudad no visitada conectada a A .
- **El Truco de la Matriz:** Cuando avanzas de i a j , en la matriz del hijo pones la fila i y columna j a ∞ , e impones $M[j][1] = \infty$ para no volver al inicio antes de tiempo. Vuelves a reducir la matriz. El coste de la reducción se suma a la cota del padre.

Traza Visual y Árbol de Exploración (El Paso a Paso)

1. El Problema (Matriz de Costes): 3 Trabajadores (A, B, C) y 3 Tareas (T1, T2, T3).

Trabajador	T1	T2	T3	Mínimo Absoluto
Ana (A)	9	2	7	2
Bob (B)	6	4	3	3
Cris (C)	5	8	1	1

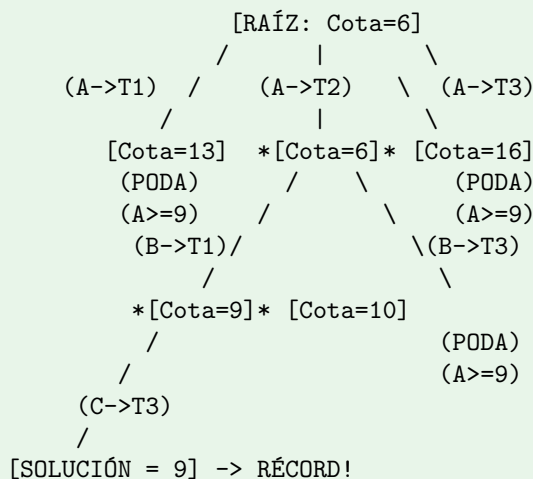
2. Secuencia en Lenguaje Natural (La Traza):

- **Paso 0 (Raíz):** Nadie tiene tarea. Cota Optimista = Suma de mínimos = $2 + 3 + 1 = 6$. Cola: [Raíz (6)].
- **Paso 1 (Ramificar Nivel 1 - Ana):** Extraemos la Raíz. Generamos 3 hijos dándole a Ana cada tarea.
 - **Hijo 1 (A→T1):** Ana cobra 9. Quedan libres T2 y T3. Mínimo de Bob (T2,T3) = 3. Mínimo de Cris (T2,T3) = 1. Cota = $9 + 3 + 1 = 13$.
 - **Hijo 2 (A→T2):** Ana cobra 2. Quedan libres T1 y T3. Mínimo de Bob (T1,T3) = 3. Mínimo de Cris (T1,T3) = 1. Cota = $2 + 3 + 1 = 6$.
 - **Hijo 3 (A→T3):** Ana cobra 7. Quedan libres T1 y T2. Mínimo de Bob (T1,T2) = 4. Mínimo de Cris (T1,T2) = 5. Cota = $7 + 4 + 5 = 16$.

Cola de Prioridad: [Hijo 2 (6), Hijo 1 (13), Hijo 3 (16)].

- **Paso 2 (Ramificar Nivel 2 - Bob):** Extraemos el mejor (Hijo 2). Ana ya tiene T2. Le damos a Bob las tareas libres (T1 y T3).
 - **Hijo 2.1 (B→T1):** Llevamos Ana(T2)=2 + Bob(T1)=6. Queda libre T3. Mínimo Cris(T3)=1. Cota = $2 + 6 + 1 = 9$.
 - **Hijo 2.2 (B→T3):** Llevamos Ana(T2)=2 + Bob(T3)=3. Queda libre T1. Mínimo Cris(T1)=5. Cota = $2 + 3 + 5 = 10$.
- Cola de Prioridad: [Hijo 2.1 (9), Hijo 2.2 (10), Hijo 1 (13), Hijo 3 (16)].
- **Paso 3 (Llegada a Meta):** Extraemos Hijo 2.1 (Cota 9). Solo queda Cris y la T3. Llevamos $2 + 6 + 1 = 9$. ¡Es una solución completa! **Récord = 9**.
- **Paso 4 (La Guillotina):** Extraemos Hijo 2.2 (Cota 10). Como $10 \geq 9$ (nuestro récord), **SE PODA**. Extraemos Hijo 1 (13) → **SE PODA**. Extraemos Hijo 3 (16) → **SE PODA**. ¡Fin del algoritmo!

3. Grafo del Árbol (Estado Final):



8. Guía de Asociación Rápida: De la pregunta a la respuesta

Para resolver cualquier diseño de Branch and Bound, aplica esta tabla:

Si el problema es de...	Tu Cota Optimista será...	Estrategia de la Cola...
Mochila / Llenado (Max)	Voraz fraccional (ordenado por b_i/p_i)	Max-Heap (Extraer mayor cota)
Rutas / Grafos (Min)	Matriz de costes reducida	Min-Heap (Extraer menor cota)
Asignación / Pares (Min)	Suma de mínimos por filas no asignadas	Min-Heap (Extraer menor cota)

El Detalle que da el 10 en el Examen

Nunca olvides mencionar cómo actualizas la **Cota Pesimista Inicial**. En lugar de inicializarla a ∞ , dile al profesor: "Para optimizar la poda desde el primer nivel, calculo una solución heurística rápida mediante un algoritmo Voraz, y uso ese valor real como Cota Pesimista inicial". Esto reduce radicalmente el tamaño del árbol de exploración en la práctica.

Apéndice: Catálogo Universal de Fórmulas RyP

Usa este catálogo para estructurar rápidamente los apartados (b), (c) y (d) de cualquier ejercicio de examen. Asumimos un nodo x en el nivel k de un problema de tamaño N .

Arquetipo 1: Selección Binaria / Recubrimiento (MINIMIZAR)

Aplicación: Cámaras en el campus (Ej 1), La pizza de Enzo (Ej 2), Aeropuertos (Ej 3), Menús del banquete (Ej 6).

Formulación Matemática: Arquetipo 1

- **a) Representación:** $x = (x_1, x_2, \dots, x_k)$ donde $x_i \in \{0, 1\}$ indica si el elemento i se selecciona (1) o no (0).
- **b) Coste Real $c(x)$:** Suma de los elementos o costes de los objetos ya seleccionados formalmente hasta el nivel k :

$$c(x) = \sum_{i=1}^k w_i x_i \quad (\text{Si todos valen igual: } c(x) = \sum_{i=1}^k x_i)$$

- **c) Coste Estimado $\hat{c}(x)$ (Cota Optimista):** Lo que llevas gastado más una subestimación del futuro $h(x)$:

$$\hat{c}(x) = c(x) + h(x)$$

- **Estrategias típicas para $h(x)$:**

- *Grafos:* Número máximo de aristas remanentes totalmente independientes (que no comparten vértices).
- *Conjuntos:* $\lceil (\text{Elementos por cubrir}) / (\text{Máxima capacidad de cobertura de un elemento libre}) \rceil$.

- **d) Regla de Poda:** Sea U el récord de la mejor solución completa (Cota Pesimista):

$$\text{Podar si: } \hat{c}(x) \geq U$$

Arquetipo 2: Emparejamientos y Rutas con Matrices (MINIMIZAR)

Aplicación: Viajante de Comercio / TSP, Asignación de Tareas planteado como mínimo (Ej 4 / Ej 7).

Formulación Matemática: Arquetipo 2

- **a) Representación:** $x = (x_1, x_2, \dots, x_k)$ donde x_i indica el índice asignado (tarea, ciudad o destino) en la etapa i .
- **b) Coste Real $c(x)$:** Suma de los pesos de las celdas seleccionadas en la matriz de costes M :

$$c(x) = \sum_{i=1}^k M[i][x_i]$$

- **c) Coste Estimado $\hat{c}(x)$ (Cota Optimista):** Lo acumulado más el coste de elegir lo mínimo absoluto que quede libre por cada fila restante:

$$\hat{c}(x) = c(x) + \sum_{j=k+1}^N \min_{l \in \text{Disponibles}} M[j][l]$$

*Nota para TSP: En el Viajante, $\hat{c}(x)$ equivale directamente al coste acumulado del padre más la arista elegida más la suma de los factores de reducción de la nueva matriz del hijo.

- **d) Regla de Poda:** Sea U el coste del viaje o asignación más barata completa descubierta:

$$\text{Podar si: } \hat{c}(x) \geq U$$

Arquetipo 3: Selección de Subconjuntos (MAXIMIZAR)

Aplicación: Mochila 0/1, Asignación de Beneficio Máximo sin cambio de signo, Impacto Visual (Ej 8).

Formulación Matemática: Arquetipo 3

- **a) Representación:** $x = (x_1, x_2, \dots, x_k)$ donde $x_i \in \{0, 1\}$ (1 si se incluye el elemento, 0 si se descarta).
- **b) Beneficio Real $c(x)$:** Suma de las utilidades o ganancias obtenidas por las elecciones realizadas:

$$c(x) = \sum_{i=1}^k b_i x_i$$

- **c) Beneficio Estimado $\hat{c}(x)$ (Cota Optimista):** El beneficio actual más una sobreestimación idealista (asumiendo que los recursos restantes se pueden fraccionar de forma óptima/voraz):

$$\hat{c}(x) = c(x) + \text{Estimación Voraz}$$

- **d) Regla de Poda: ¡OJO! Al maximizar se invierte la guillotina.** Sea U el beneficio de la mejor solución real completa que ya tenemos en el bolsillo. Si tu máxima esperanza del futuro no supera la realidad actual, se corta:

$$\text{Podar si: } \hat{c}(x) \leq U$$